Practice Questions for the second midterm
Winter 2026

1. [8 points] **Dynamic Programming**

   You are given an array *prices* where $prices[i]$ is the price of a given stock on the $i-th$ day.

   You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

   Return the $maximumprofit$ you can achieve from this transaction. If you cannot achieve any profit, return 0.

   Example 1:

   Input: prices = [7,1,5,3,6,4]

   Output: 5

   Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

   Example 2:

   Input: prices = [7,6,4,3,1]

   Output: 0

   Explanation: In this case, no transactions are done and the max profit = 0.

   (a) What is the brute-force solution? Mention and analyze the runtime.

   (b) What is the recursive formula?

   (c) develop the pseudocode using memoization and bottom-up technique.

2. [8 points] **Dynamic Programming**

   For the following recurrences formula, which is the solution to a problem solved using dynamic programming,

$$M(i) = \begin{cases} 1 & \text{if i == 1} \\ min_{1 \le j \le i}(M(i)/i + w(i)) & \text{Otherwise} \end{cases}$$

   (a) give the time and space that would be required by a simple dynamic programming algorithm to compute the answer, $M(n)$, where the values of the w function are given as input.

   (b) Write the bottom-up implementation of the above recursive formula.

3. [8 points] **Greedy**

   You are given N positive integers, and $N-1$ signs. Each of the signs is either a plus ('+') or a minus ('-'). You need to place the numbers in a sequence and then put a single sign between each pair of integers. The goal is to get the **maximum value** when we evaluate an expression. You can place the signs and the numbers in any order. Please write a few

words of your approach followed by a pseudocode algorithm and provide time complexity for it. Prove the optimality of your algorithm.

Consider the following example:

Numbers: 5, 7, 9, 3

Signs: +, +, -

Output: [5+7+9-3 = 18]

Hint: (If you sort, you don't need to write sorting algorithm, you can use the time complexity of O(n lg n))

4. [8 points] **Runtime Analysis**

   Consider the following recurrence

$$T(n) = T(n-1) + T(n-2) + O(1) T(0) = T(1) = 1$$

   Which is the recurrence for fibonacci number. In class, by expanding the reurrence relation we showed that $T(n) = O(2^n)$. Use induction to prove that $T(n) = O(2^n)$.